

Viva Question Bank, with Answers

AI-Powered Legal Document Analysis System: Clause Detection, Category-Level Risk
Prioritization and Plain-Language Explanation on CUAD

Group ID P25301051 Jerin Aktar Afnan Mazumdar Shoyeb Hasan Sayem
Supervisor: Utsha Kumar Roy
BRAC University, Department of Computer Science and Engineering

How to use this document

Part A answers the eight core questions in full. Parts B to J are the wider question bank, grouped by topic. Part K is a one-page number sheet to memorise.

Each long answer has two layers:

- **Say this:** the 20 to 40 second spoken answer.
- **If pushed:** the detail for when the examiner says “and why?” or “prove it”.

Every number here comes from the final report (October 2026) and its result files. The project’s number checker, `retrain/verify_report.py`, compares 230 of them with those files.

The project has **two stages of results**, and you must keep them apart:

- **First setup:** the presense transformer read 256 tokens per window and nothing was tuned. Here the TF-IDF baseline beat the transformer (0.775 against 0.692).
- **Re-run** (report Section 5.3.3): the transformer reads the whole window (512 tokens) and both models are tuned on 81 held-out training contracts. Here the transformer ties the baseline (0.797 against 0.785) and the ensemble beats it (0.809). The app uses the re-run models.

When asked “which model won?”, always say which stage you mean.

Part A, The eight questions

A1. Where did we find the dataset?

Say this: We used CUAD, the Contract Understanding Atticus Dataset, built by The Atticus Project and published by Hendrycks and colleagues at the NeurIPS 2021 Datasets and Benchmarks track. We downloaded the official release from the Hugging Face hub, [theatticusproject/cuad](https://huggingface.co/theatticusproject/cuad). It has 510 commercial contracts from public US SEC filings, and experienced lawyers marked every clause that belongs to one of 41 categories. The labelling is reported to be worth about two million US dollars of lawyers' time, which is why we could not build anything like it ourselves.

If pushed, the detail that shows you handled the data: The default `load_dataset()` call gives only the 511 raw contract PDFs, which cannot be trained on. The labelled data is in two files:

- `master_clauses.csv`: a wide table, 83 columns, one row per contract. Used for exploration and to double-check the labels.
- `CUAD_v1.json`: the SQuAD-format file with full contract text and character offsets of every answer. All modelling uses this file.

The wide CSV becomes $510 \times 41 = 20,910$ (contract, category) rows, 32.1 percent positive. Two decisions were not obvious. We paired each clause-text column with its answer column **by position, not by name**, because one header, “Notice Period To Terminate Renewal- Answer”, has an extra space that breaks matching by name. And a category counts as present when its clause-text list is non-empty, not from the Yes/No column, because eight categories (dates, party names) hold a value there instead of Yes or No.

Likely follow-up, “why not collect your own contracts?” Labelling legal text needs lawyers, and lawyers are expensive; that is why the field was stuck before CUAD. Three students with one laptop and one semester could not label even 20 contracts to this standard.

A2. Which models did we use?

Say this: Three trained models plus a baseline, a fixed risk table and a calibration map. In pipeline order:

1. **TF-IDF plus logistic regression**, the presence baseline: 20,000 word and word-pair features, one class-balanced logistic regression per category.
2. **DistilBERT, sequence-classification head**, the window-based presence model. It scores a (category question, window) pair, and the contract’s score is the highest window score. In the final system it reads the whole 2,000-character window (512 tokens).
3. **Ensembles** of the two. In the re-run the rule and its per-category thresholds were chosen on a validation split: AND for the best overall F1, averaging for the recall-first setting.
4. **DistilBERT, question-answering head**, the span extractor, which predicts where the clause starts and ends.
5. **FLAN-T5-small**, the plain-language step, fine-tuned on 41 category sentences we wrote.

On top sits the **risk table**, deliberately not a model: 41 categories mapped to High, Medium or Low (8 / 22 / 11), each with a one-line reason. The app also uses an **isotonic calibration map** fitted on the validation contracts, so the confidence it shows is a real chance.

If pushed, “why so small?” Everything trained on one Apple M4 laptop with 16 GB of memory. Our first choice, DeBERTa-v3, gave NaN gradients on Apple’s MPS backend and took about 89 seconds per step on the CPU, so we switched to DistilBERT (67 million parameters). The method stays the same; on a CUDA machine the model name can be changed back. Our scores are therefore not comparable with the published CUAD results, which use much larger DeBERTa models, and we never present them as comparable.

A3. How did we implement those models?

Say this, walk the pipeline:

Presence. Each contract is cut into 2,000-character windows that move forward 1,500 characters at a time, so neighbours overlap by 500 characters. For each of the 41 categories we pair the category’s CUAD question with every window, and the classifier says whether that window contains a clause of that category. The contract’s score is the **maximum over all windows**: multiple-instance learning, where the contract is a bag of windows and “the clause is somewhere in this contract” means “at least one window is positive”. Training uses 24,000 window examples, DistilBERT, learning rate 2e-5, batch 16, on MPS.

The re-run. In the final model the input limit is 512 tokens instead of 256, so the whole window is read. We held out 81 of the 408 training contracts as a validation split, trained on the other 327 for three epochs, and kept the epoch with the lowest validation loss (the second; losses 0.252, 0.245, 0.259). On the validation contracts we chose, for both models in the same way, the TF-IDF regularization, the pooling rule, one threshold per model and then a threshold per category (for categories with at least eight validation positives), and the ensemble rule. Each choice was made twice: once for micro-F1 and once for micro-F2, which treats recall as twice as important. Then the test set was scored once.

The baseline. TF-IDF has no length limit, so it reads the whole contract. One logistic regression per category; trains in under a minute.

Span. DistilBertForQuestionAnswering, input (CUAD question, window), output start and end token positions. Character offsets become token labels through the tokenizer’s offset mapping. 8,000 of 11,156 examples, 320 tokens, learning rate 3e-5, about 25 minutes.

Plain language. We wrote 41 template sentences, one per category from the weaker party’s side, paired each clause with its category’s sentence (11,156 pairs), and fine-tuned FLAN-T5-small on `summarize in plain English: <clause>`. 4,000 examples, learning rate 3e-4, on the CPU.

If pushed, “why two window sizes?” They do different jobs. The 2,000-character window is a **presence** window that tiles the whole document, because at that point we do not know where anything is. The 1,200-character window of the span step is a **focus** window, placed so the answer starts about 150 characters in, so that the answer always fits inside the 320-token limit during training.

The leakage question hiding inside that. Placing the focus window uses the answer’s position, which is part of the label. That is allowed only because it happens at training time: it decides which text becomes a training example and never what the model reads when the app runs. Our rule: labels may shape training targets, never an input or a choice made at inference.

A4. Why did we use these models?

Why TF-IDF plus logistic regression? Legal drafting is formulaic (“This Agreement shall be governed by the laws of...”), so the words alone carry a lot of signal, and a bag-of-words model has no length limit, so it reads the whole contract. We built it first as the score every neural model had to beat. In the first setup it looked unbeatable; in the re-run the tuned transformer ties it.

Why a window-based transformer, and why multiple-instance learning instead of retrieval? Because we measured retrieval first. The obvious design scores windows against the category and gives the model only the best few. The best search we tried (category name, top 3 windows) contained the clause only **64.0 percent** of the time. No model after the search can beat that, so a retrieval design would lose before training. The measurement took seconds.

Why DistilBERT? DeBERTa-v3 failed on MPS; DistilBERT trains on the laptop; and the app has to answer in about a minute on a CPU.

Why not Longformer? We measured it. Longformer-base at 4,096 tokens on our M4: one training step takes **258 seconds** and 14.71 GB at batch size 1, so our training schedule would take about **136 days**. Scanning one median contract takes about **454 seconds** with Longformer against **10.6 seconds** with DistilBERT. And 4,096 tokens is still only about half of a typical contract, so it would need windows anyway. Be honest about one correction: we first believed Longformer would not fit in memory at all. It does at batch size 1; time is the real limit.

Why FLAN-T5-small? Instruction-tuned, small enough to fine-tune on a CPU. We report that this step did not work as a summarizer (A6, Part G).

Why a lookup table for risk? It can be checked line by line, every level has a written reason, and it cannot change silently between versions. We had no risk-labelled data to learn from. The cost: it cannot tell a \$10,000 liability cap from a \$10,000,000 one.

A5. How do the models work?

The TF-IDF baseline. Every contract becomes a 20,000-dimensional vector: how often each word or word pair appears, weighted down when the term is common everywhere. A logistic regression per category turns that into a probability. No word order, no meaning, only which terms appear.

The window-based DistilBERT and the MIL rule. DistilBERT is a 6-layer transformer that reads a pair of texts and outputs present or absent. A median contract has 22 windows, so 41 categories mean about 900 forward passes per contract. Then:

$$\text{score}(\text{contract}, \text{category}) = \max \text{ over windows of } P(\text{window contains this clause})$$

Max is right in principle: the contract contains a non-compete if at least one window does. Its cost is that **a maximum over dozens of windows is dozens of chances to be wrong**, so the scores bunch near 1. That is why a 0.5 threshold suited it badly, and why the thresholds chosen on validation are high (0.96 for micro-F1, 0.88 for micro-F2).

Why 512 tokens mattered. The CUAD question alone takes a median of 53 tokens. At 256 tokens only about 200 were left for the window, so the first model read a median of **49.3 percent** of each window (about 986 characters), about a third of every contract was never read, and **23.2 percent** of positive training windows had the clause after the cut. At 512 tokens the whole window fits.

The 500-character overlap. The median clause is 196 characters, so a 500-character overlap means every clause fits completely inside at least one window.

The ensembles. The two models make different mistakes: the transformer over-detects, the baseline is more careful. AND reports a clause only when both pass their thresholds, so a false alarm survives only if both make the same mistake. Averaging is softer and keeps more recall.

The span model. Same backbone, different head: two distributions over token positions, start and end. We take the best start and end inside the context, decode the tokens between them, and cap the length.

The summarizer. A sequence-to-sequence model: prompt and clause in, one sentence out. What it actually learned is in A6 and Part G.

The calibration map. An isotonic regression fitted on the 81 validation contracts. It maps the transformer’s raw score to the share of contracts with that score that really contained the clause. It keeps the order of the scores, so it changes no detection, only what the app displays.

The risk layer. A dictionary lookup: category in, level and reason out; results sorted High, Medium, Low.

A6. Future work, in short

Say this: What is left of the end-to-end loss is split about evenly between the window the presence model chooses (206 clauses) and the span model inside the right window (244), so both come first, then users.

1. **Train a model to choose the window.** We already retrained the span model on the chunks it really reads (end-to-end 22.1 to 59.6 percent). What is left is mostly the window: the presence model’s best window misses the clause for 17.4 percent of detected categories. A locator trained for that job, or the new span model scoring several candidate windows (it can now say “not here”), targets it. The span model itself still misses 23.6 percent in the right window; it used 6,000 clauses and one seed, so more data and more seeds come next.
2. **Watch people use Clausify.** Five to ten non-lawyers, with and without the app, measuring time, clauses found, and above all whether they trust the output too much. The protocol is written (USABILITY_PROTOCOL.md).
3. **Judge each clause, not just its type:** read the actual amount of a cap or the length of a non-compete.
4. **A summarizer that really summarizes:** grow our 150 clause-specific references into a few thousand targets, or use a larger instruction-tuned model, and rate faithfulness by hand.
5. **Tune for risk directly,** with a goal that weights High-risk categories more.
6. **Cross-validation:** the split was the largest uncertainty we measured.
7. **A trained Longformer and larger models on a CUDA machine.**
8. **Contracts from outside CUAD,** starting with Bangladeshi contracts.

Already done in the re-run, so do not list them as future work: the validation split, choosing thresholds and the ensemble rule on it, recall-first tuning, and calibration.

A7. Industrial use of the project

Say this: The target user is someone who signs a contract without a lawyer reading it first: small businesses, start-ups, freelancers. Four uses:

1. **First look before signing:** which of 41 clause types are in a contract and which three or four deserve a lawyer’s half-hour. It points to the page; it does not replace the lawyer.
2. **Due-diligence support:** a first pass over hundreds of contracts for the same provisions, with a person checking every result.
3. **Procurement and contract management:** flag auto-renewals, uncapped liability, exclusivity and minimum commitments across many supplier contracts.
4. **Legal-access tools where lawyers are scarce,** the Bangladeshi setting that motivated the work.

The deployment property worth stating: Clausify runs locally on small models, on a CPU, about a minute for a median contract and about 2.1 GB of memory. The contract never leaves the computer. The privacy comes from where it runs, not from anything clever in the app.

The honesty clause, say it unprompted: We would not ship this yet. The quoted clause text is unreliable end to end (about 22 percent of real clauses get through all three steps), so a card means “a clause of this type seems to be here, look”, not “this is the clause”. The risk level is per category. The badge and the category name are the trustworthy parts.

Likely follow-up, “who are the competitors?” Kira, Luminance, eBrevia: enterprise-priced, cloud-based tools for legal professionals. Ours is local and risk-sorted for a signer, and weaker on accuracy, coverage and jurisdictions.

A8. Is there a dataset more advanced than CUAD? Why not use it?

Say this: Newer legal datasets exist, but none replaces CUAD for this task: lawyer-marked character spans for 41 clause categories over whole, unsegmented commercial contracts.

Dataset	What it is	Why we did not use it
MAUD (EMNLP 2023)	Expert answers to 92 questions over 152 merger agreements	One contract type, multiple-choice reading comprehension, not span finding over general contracts
ContractNLI (EMNLP Findings 2021)	607 NDAs, 17 hypotheses, entailed / contradicted / not mentioned, with evidence spans	NDAs only, and document-level inference, not clause finding; a good future test
LEDGAR (LREC 2020)	Hundreds of thousands of already-segmented provisions	Removes the hard part, finding the clause inside a 33,000-character contract
LexGLUE (ACL 2022)	Seven legal understanding tasks	A benchmark suite, none of it span extraction over contracts
LegalBench (2023)	Legal reasoning tasks for testing large language models	An evaluation suite, not a training corpus of the kind we needed
CLAUDETTE / Unfair-ToS	Terms-of-service clauses labelled for possible unfairness	The closest public thing to a risk label, but consumer terms, not negotiated contracts

The stronger point: no public dataset labels **how serious** a clause is. CUAD says a non-compete is present, not how harsh it is. That is why we wrote the 41-level risk table ourselves.

If pushed, “would more data have saved the first transformer?” No: training on all 53,662 windows instead of 24,000 changed micro-F1 by -0.0004. What helped was reading the whole window.

Part B, Dataset and preprocessing

B1. How many contracts, categories, labelled examples? 510 contracts, 41 categories, 20,910 (contract, category) rows, 32.1 percent positive; 408 train, 102 test. In the re-run, 81 of the 408 are held out for validation.

B2. Why 510 and not 511? The repository has 511 PDFs; the labelled data covers 510 contracts.

B3. How did you split, and why? 80/20 by whole contract, seed 42, so no clause from a training contract appears in the test set. Splitting by row would let the model memorise a contract’s drafting style. The same split is used by every model.

B4. Is the split balanced? 32.0 percent positives in training against 32.2 percent in testing; over all 41 categories Pearson $r = 0.986$, average difference 3.4 points. One outlier: Termination for Convenience, 47.1 percent of test contracts against 33.1 percent of training contracts. We report it instead of redrawing the split.

B5. Is your split a lucky one? No. Over 20 random splits the baseline averages 0.7829 (SD 0.0063); ours gives 0.7747, below the mean.

B6. What is seed 42? The fixed starting value for every random step (split, shuffling, start weights, the 150 summarizer clauses), which makes the project repeatable. 42 is the conventional default, chosen without searching. Searching for a lucky seed would be tuning on the test set.

If pushed, “was 42 lucky for the transformer?” In the first setup we trained it with seeds 42, 43, 44: micro-F1 0.6924, 0.6928, 0.7082 (SD 0.0090); the baseline’s lead of 0.0824 is 9.2 SDs, and even the luckiest seed lost by 0.0665. That is why the real cause, the 256-token cut, had to be something every run shared.

B7. How long are the contracts? Median 33,143 characters (about 5,000 words), longest 338,211. About 16 times what a transformer can read at once; most design choices follow from this.

B8. How long are the clauses? Median 196 characters, 90th percentile 587, 99th 1,330. A clause fits in one window; finding it is the hard part.

B9. How bad is the imbalance? Document Name is in 100 percent of contracts; Source Code Escrow in 2.5 percent (13 contracts). The median category appears in about 23 percent.

B10. How did you handle it? Class-balanced logistic regression; two negative windows per contract and category, keeping about 39 percent positives; and we report macro-F1 over the 34 categories with at least ten test positives, where the re-run transformer scores 0.711 against 0.673 for the tuned baseline.

B11. What preprocessing? Very little. Text used as is; WordPiece handles casing and pieces; TF-IDF uses sublinear term frequency and a minimum document frequency of 2.

B12. Data augmentation? No. Rewriting legal text can change its legal meaning and corrupt the label.

Part C, Model and method

C1. Multiple-instance learning in one sentence? Labels belong to a bag of items, not to each item: the contract is the bag, windows are the items, and a positive bag has at least one positive window.

C2. Why max-pooling and not the mean? Max is the multiple-instance rule; the mean of 35 windows, one positive, cannot reach a high score.

C3. Did you test other pooling rules? Six, on the first-setup model. At 0.5 the top-3 mean beat the max (0.7115 against 0.6924); with each rule at its own best threshold, max was best (0.7529). None reached the baseline. In the re-run, max was also the rule chosen on validation.

C4. Why was 256 tokens a mistake? Because the question uses about 53 tokens, so the model read only about half of each 2,000-character window, about a third of every contract was never read, and 23.2 percent of its “present” training examples pointed at text it never saw. Every check in the first setup (more data, seeds, pooling, thresholds) kept the same cut, which is why none of them found it.

C5. How do you know the 512-token fix, and not something else, made the difference? A control run. We repeated the whole re-run with the input left at 256 tokens: same 327 contracts, same validation split, same tuning. Untuned, it scores 0.693, the same as the original 0.692, so the change of training contracts does nothing. Reading the whole window adds +0.024 micro-F1 untuned (interval +0.014 to +0.035) and +0.036 when both are tuned (+0.021 to +0.051). At 256 tokens the tuned transformer still loses to the tuned baseline by 0.024.

C6. How was the ensemble chosen? In the first setup the rules (AND, OR, average) had no settings at all, so nothing could be fitted to the test set. In the re-run we chose the rule and thresholds on the validation split: AND for micro-F1, averaging for micro-F2. A mixed rule (OR for High-risk categories, AND for the rest) was a candidate and was not chosen.

C7. Is the ensemble’s win real? In the first setup, no: +0.0042 over the baseline, interval [-0.0037, +0.0115], a tie. In the re-run, yes: +0.024, interval [+0.011, +0.037], about 3.8 times the split-to-split SD. The control run shows this lead comes from tuning; it is the same with 256 tokens (0.810).

C8. Why cluster bootstrap? The 4,182 decisions come from 102 contracts, 41 each, so they are linked. Resampling whole contracts gives honest intervals; for span token-F1 the contract interval [0.743, 0.782] is about twice as wide as the clause interval [0.752, 0.775].

C9. Why not use an LLM like ChatGPT? Cost and repeatability (every number must be reproducible from files on disk), privacy (the contract must not leave the computer), and scope (the question was whether a small local pipeline can do this). We used an LLM assistant for one declared job: drafting the 150 clause-specific reference sentences for the summarizer test, from clause text only.

C10. DistilBERT vs BERT? A 6-layer distilled BERT, 67M against 110M parameters, about 60 percent faster with most of the accuracy.

Part D, Training

D1. Hardware? One Apple M4 laptop, 16 GB, MPS backend, torch 2.8, transformers 5.x.

D2. How long did training take? First setup: presence about 50 minutes, span about 25, summarizer 8 to 13 on the CPU. The re-run presence model at 512 tokens: 191 minutes to train for three epochs and 198 minutes to score every window of the validation and test contracts.

D3. Hyperparameters, and how chosen? Standard fine-tuning defaults: learning rates 2e-5, 3e-5, 3e-4; batch 16, 16, 8; two epochs. In the first setup nothing was tuned. In the re-run the presence model’s epoch, pooling rule and thresholds, the baseline’s regularization and the ensemble rule were chosen on the 81 validation contracts, never on the test set.

D4. Was “we did not tune” a strength or a weakness? Both, and it turned out to be the most serious threat in the project. It kept the test set clean, but the transformer competed with default settings and half its window cut off. The re-run answers it by tuning both models the same way.

D5. Optimizer and schedule? AdamW, linear decay, no warm-up, gradient clipping at 1.0, read back from the saved `training_args.bin`.

D6. No warm-up, is that safe? For short runs from pre-trained weights, mostly. But clipping was active on almost every step (presence gradient norms 1.45 to 10.60, median 4.30), so part of what kept training stable was the clipping.

D7. What validation did you use? First setup: 5 percent of the training windows, for monitoring only. Re-run: 81 whole training contracts, used to choose the epoch, pooling, thresholds and ensemble rule, and to fit the calibration map.

D8. Did you ever touch the test set during development? Once, wrongly: an early exploratory run of the summarizer monitored 300 test clauses. That run kept no checkpoint and chose nothing, and the summarizer used for every result was retrained with no evaluation set. We report it.

D9. Overfitting? First setup: training loss 0.57 to 0.26 and validation 0.275 to 0.254, close together. Re-run: validation loss lowest after the second of three epochs (0.245), so we kept it.

D10. What went wrong? DeBERTa-v3 NaN gradients on MPS; MPS memory not freed between runs (summarizer moved to CPU); during the re-run, memory grew into swap until we padded batches to a few fixed lengths; library changes in transformers 5.x.

Part E, Results and evaluation

E1. Headline numbers? Presence, re-run: the tuned ensemble reaches micro-F1 **0.809** and accuracy **88.43 percent**; the app’s recall-first setting finds **90.3 percent** of High-risk clauses. Span, retrained: token-F1 **0.779** on any window that holds the clause, and **59.6 percent** of real clauses get a good quote end to end (22.1 with the first span model).

E2. Which model won? First setup: the TF-IDF baseline (0.775) beat the transformer (0.692), and the AND ensemble (0.779) tied the baseline. Re-run: the transformer ties the tuned baseline (0.797 against 0.785; lead +0.012, interval [-0.005, +0.030]), is better on macro-F1 over the common categories (0.711 against 0.673), and the tuned ensemble beats the baseline (0.809).

E3. Why did the transformer lose in the first setup? Two flaws in our own setup: it read only about half of each window, and it was judged at a threshold of 0.5 that its scores did not suit. The first-setup checks ruled out data (-0.0004 with all 53,662 windows), seed (gap 9.2 SDs) and pooling. Fixing both flaws closed the gap; the control run shows neither fix alone is enough.

E4. Why is a bag-of-words model so strong here? Legal wording is formulaic, and it reads the whole contract while the transformer reads windows and has to combine them.

E5. Is micro-F1 the right metric? No. Half of the real clauses are Low-risk (dates, names, governing law). In the first setup the best micro-F1 setup, the AND ensemble, missed **36.9 percent** of High-risk clauses, while the transformer alone missed 15.9 percent and OR 14.2 percent. In the re-run we fixed a recall-first goal (micro-F2) in advance and chose on validation: the app’s setting misses **9.7 percent** (17 of 176). The F1-tuned ensemble still misses 46.0 percent, the same conflict.

E6. Why not just switch to OR in the first setup? Choosing it after seeing the test set would have been test-set tuning. The re-run did it properly, on a validation split.

E7. End-to-end performance? With the app’s setting and the retrained span model, **59.6 percent** of the 1,348 real clauses get through all three steps (interval 57.0 to 62.2). With the first span model it was 22.1 percent (19.7 to 24.6), and 22.0 with the first setup, when the separate scores predicted about 77 and 65 percent.

E8. Where does it fail? There were two causes. Window choice: the best-scoring window misses the clause 17.4 percent of the time with the re-run model, against 30.9 percent before, so reading the whole window fixed most of this, and it is now about as large a loss as the span model’s. Span model: with the right window the first span model found the clause only 28.0 percent of the time (39.6 percent with the first setup), because it was trained with the answer always near the start of a centred window. The data showed it: when the clause sat in the second half of the window it succeeded only 5.4 percent of the time (about 41 percent in the first half), and the first presence model rarely chose such windows because it never read that half.

If pushed, “so why didn’t you fix it?” We did. We retrained the span model on exactly the 1,200-character chunks it reads in the app, with the answer wherever it falls and “no answer” chunks included (SQuAD 2.0 style), on the 327 fit contracts; epoch and decoding were chosen on the 81 validation contracts. On the test set it finds the clause in the right window 76.4 percent of the time (28.0 before), 76.2 in the first half and 76.8 in the second, and end-to-end survival rises from 22.1 to 59.6 percent. Training took 46 minutes. Of the 545 clauses still lost, 95 are never detected, 206 are outside the chosen window and 244 are missed inside it.

If pushed, “is the quoted paragraph any good?” Measured now: the app quotes the paragraph around the new model’s answer, and on the test set it contains at least half of the clause for 77.2

percent of detected clauses (71.7 percent of all real clauses). The old method, the paragraph the presence model scored highest, reached 75.3 percent; we picked between them on validation.

E9. Why is exact match so much lower than token-F1? Legal clause edges are debatable (should the section number be included?). For the first span model on centred windows, 82.8 percent of answers overlap the real clause at F1 0.5 or more; that 0.764 / 82.8 percent was a best case, and on plain windows the same model reached only 0.366. The retrained model reaches 0.779 on plain windows.

E10. Comparable with published CUAD results? No: their models are about ten times larger, and they use a full-document ranked test; ours measures extraction inside windows that contain the answer.

E11. Is the model calibrated? The raw score is not: Expected Calibration Error 0.206 on the test set. An isotonic map fitted on the validation contracts brings it to **0.016** (Brier 0.195 to 0.091; Platt scaling 0.084). The app shows this calibrated chance, limited to 1 to 99 percent.

E12. Rare categories? First setup: the AND rule gained nothing on the ten rarest (F1 0.364 against 0.368) and found only 18 of 70. Re-run: the recall-first transformer finds 49 of 70 (F1 0.460) and scores zero on only one of the ten; the balanced AND setup still finds 18 and scores zero on five.

E13. How much uncertainty? Three sources in the first setup: bootstrap half-width about 0.008, seed SD 0.0015 (ensemble), split SD 0.0063. The first setup's +0.0042 lead was inside them; the re-run's +0.024 lead is outside the bootstrap interval and 3.8 split SDs.

Part F, Risk table

F1. How did you assign the levels? Five questions from the weaker party's side: can it cost money without a limit; can it be undone; does it limit what you can do later; is it one-sided; does it last after the contract. **High:** no money limit, cannot be undone, or limits you later. **Low:** only records a fact or gives a benefit. **Medium:** everything else.

F2. A non-obvious example? Cap on Liability is Medium, not Low: it limits what you can recover. Governing Law is Low: it is a fact about the contract, not a one-sided duty.

F3. The eight High categories? Money you could owe without limit: Uncapped Liability, Liquidated Damages, Minimum Commitment, Most Favored Nation. Something you cannot get back: IP Ownership Assignment, Irrevocable or Perpetual License. Your freedom to earn later: Non-Compete, Exclusivity.

F4. Who is the “weaker party”? The party with less say over the wording, usually the one who did not write the contract; where unclear, the party who carries the duty. A clause that is High-risk for one side often protects the other, and the app states its assumption.

F5. Why should anyone take your table seriously? Because it can be checked: every level has a written reason against five named rules, so anyone can disagree with a specific level for a specific reason, which is not true of a learned score.

F6. Is category-level risk good enough? No, and that is why we call it category-level risk prioritization. It says “contracts with a non-compete deserve attention”, never “this non-compete is harsh”. It cannot tell a \$10,000 cap from a \$10,000,000 one.

F7. Does jurisdiction matter? Yes. A non-compete binding in one place may be void in another. CUAD is US contracts only; we have no evidence for Bangladesh.

Part G, Summarizer

G1. What does it do? Honestly, it does not summarize. It picks one of 41 category sentences.

G2. How do you know? We tested it on 150 test clauses against two reference sets: our 41 templates and 150 clause-specific references. ROUGE-L 0.775 against the templates, **0.116** against the clause-specific references, and the bare template with no model scores **0.115**. A lookup table matches it.

G3. What does it output? For all 150 clauses, one of the 41 templates word for word: the right one 70 percent of the time, another category’s 30 percent.

G4. Why? The training targets: 41 unique sentences across 11,156 pairs. The easiest way to lower the loss is to recognise the category and copy its sentence.

G5. Did fine-tuning help? It hurt: the untrained FLAN-T5 scores 0.299 against the clause-specific references. It mostly echoes the source, which inflates word overlap, so it is not a good summarizer either.

G6. The lesson? An automatic score measures its reference sentences as much as the task.

G7. How would you fix it? Clause-specific targets (start from our 150 and add more) or a larger instruction-tuned model, then a small human rating of faithfulness and clarity.

Part H, Clausify application

H1. What is it? A Streamlit app (`app.py`) on two modules: `model_utils.py` (models, windows, scoring, rules, calibration, risk table) and `features.py` (file reading, highlighting, reports). It runs all the models on the user’s own computer.

H2. User flow? Paste text or upload a `.pdf`, `.docx` or `.txt` file; choose Whole contract, Separate clauses or Auto; analyze. The report shows counts per risk level and one card per clause: category, risk badge and reason, a plain-English line, the calibrated chance, and the quoted paragraph with the key phrase highlighted. Below: the whole contract highlighted by risk, a missing-protection checklist, and PDF and CSV downloads.

H3. Which detection setting does it use, and why? By default the re-run transformer alone with its recall-first thresholds: it misses the fewest High-risk clauses on the test set (9.7 percent). The recall-first ensemble and the balanced AND ensemble are settings. The default does not depend on the TF-IDF model, which learnt from long SEC filings and scores short contracts too low: on our 7,033-character demo contract it scores the IP-assignment clause 0.08, so the ensemble misses it.

H4. How fast? On the M4, CPU only: shortest test contract 2.0 s, median **64.4 s**, worst **87.1 s**, peak memory 1.93 GB. Reading whole windows doubled the time of the first setup (median 30.1 s).

H5. Why do the two longest contracts take about the same time? Both hit the **30-window cap** (about 45,500 characters). 38.4 percent of CUAD contracts are longer, and clauses after the cut cannot be found. It is a limit of the app, not of the method.

H6. Can a user trust the bar? Yes, more than before: it is the calibrated chance (ECE 0.016 on the test set), measured on CUAD contracts, so on very different contracts it is only a guide.

If pushed, “why does a card say 9 percent and still report the clause?” The recall-first thresholds are set per clause type, but the calibration map is shared, so some types are reported at a low chance. On the test contracts 728 of the 1,963 default cards (37.1 percent) are under 50 percent, and 204 (10.4 percent) under 20. We keep them because they still hold 46 of the 159 High-risk clauses found; without them High-risk recall drops from 90.3 to 64.2 percent. The app now labels them **Possible** — **check** and lists them after the other cards of their risk level. Of the cards at 50 percent or more, 83.9 percent are real clauses; of the “possible” ones, 29.8 percent.

H7. Can a user trust the quoted text? Less than the badge, but much more than before. With the retrained span model, 59.6 percent of real clauses get a good extracted answer end to end (it was about a fifth), and the quoted paragraph contains at least half of the clause for 71.7 percent of real clauses. A card still means “look here”, and the app says so. On the demo contract the Liquidated Damages card quotes the limitation-of-liability paragraph; the previous quoting method found Section 13 there, but it was slightly worse on the validation and test contracts overall.

H8. What does it find on your demo contract? 27 clause types: 5 High, 13 Medium, 9 Low. The four High-risk types the section headings name (Exclusivity, IP Ownership Assignment, Non-Compete, Liquidated Damages) are all found; one High card, Uncapped Liability, is a false alarm on a clause that caps liability, and the summarizer’s warning on that card points to the right category. It misses the revenue-sharing and the 90-day warranty clauses. We kept these mistakes in the report’s figures.

H9. Clause-by-clause mode? One card per numbered clause; the clause heading decides the category when it names one, otherwise the model’s adjusted scores do (see the report for its measured accuracy). A contract with 18 numbered clauses gives exactly 18 cards.

H10. Privacy? Everything runs locally; nothing is saved or sent. On a shared server that would no longer hold.

H11. Is it tested? 45 automatic end-to-end tests, including checks that the deployed thresholds are the tested ones, that the calibration map is valid, and that every card under 50 percent is marked “Possible — check”. The output of the last run is saved in the app repository as `tests/test_output.log`.

H12. Has anyone outside your group used it? Not in a study. The usability protocol is written; the claim that a risk-sorted report helps a non-lawyer is untested.

Part I, Ethics, limitations, threats

I1. Is this legal advice? No. It sorts clause types for attention, and the app says it is not legal advice.

I2. Worst case today? A High-risk clause is present, the system misses it, and the user signs believing they were warned. Even the app’s recall-first setting misses 9.7 percent of High-risk clauses. That is why we chose recall-first for the app.

I3. Threats to validity? *Internal:* the first setup was untuned (fixed by the re-run); the re-run made many choices on only 81 validation contracts, which makes validation scores optimistic but leaves test scores fair; the summarizer’s early run saw test clauses. *Construct:* ROUGE-L does not measure summary quality here; micro-F1 does not measure harm; category-level risk is not clause-level risk; token-F1 assumes clear clause edges. *External:* 510 US contracts from SEC filings, English only. *Statistical:* three seeds is a small sample; five categories have at most seven test examples; one split; the re-run’s seed and split variation is unmeasured.

I4. Biggest limitations? No user study; small models; the presence model’s choice of window (17.4 percent wrong); the summarizer does not summarize; risk is per category; the 30-window cap; one dataset.

I5. AI tools? One declared use affecting a number: an LLM assistant drafted the 150 clause-specific reference sentences from clause text alone, with no sight of model output.

I6. Reproducible? Yes. The split, all models, the window scores and the thresholds are saved; scripts regenerate every table; `retrain/verify_report.py` checks 230 reported numbers.

Part J, Hard questions

J1. “So what did you contribute?” A complete system from contract to risk-sorted report, which CUAD work does not have; a controlled comparison showing how a strong baseline can look unbeatable when the transformer reads half of each window and is held to an untuned threshold, with a control run that separates the fixes; a risk table for the 41 categories with written rules; a recall-first, calibrated app; and measured negative results about the summarizer and about how errors add up.

J2. “0.779 against 0.775 is noise.” It was, and we said so: a tie in the first setup. The re-run’s 0.809 against 0.785 is not: interval [+0.011, +0.037].

J3. “Your first result was wrong. Why should we trust the new one?” Because we found the flaw ourselves, fixed it under a stricter protocol (validation split, test scored once), and ran a control that changes only the input length. Every choice was made on validation contracts.

J4. “22 percent end to end means it does not work.” It meant that for the first span model, and we fixed the cause: retrained on the chunks it really reads, 59.6 percent of real clauses get through end to end. Presence works: 93.0 percent of real clauses are found with the app’s setting. What is left is split between the window the presence model chooses and the span model.

J5. “You thought Longformer would not fit in memory.” We measured it and corrected ourselves: it fits at batch size 1; time is the limit (136 days).

J6. “This only works on American contracts.” Correct, and we say it first among external threats. The cheapest next step is to run a few local contracts and check them by hand.

J7. “Starting again, what would you change?” Draw the split before any measurement; set aside a validation split from day one; check how much of each input the model really reads; define the goal from the purpose (recall on High-risk clauses) before running anything.

J8. “The one sentence to remember?” Separate scores do not describe the system: a strong baseline looked unbeatable until we found the transformer was reading half of each window, and the setup that won on the overall score was the worst for the clauses the system exists to warn about.

Part K, Number sheet

Dataset

Source	CUAD v1, NeurIPS 2021 D&B, HF theatticusproject/cuad
Contracts / categories	510 / 41; 20,910 rows, 32.1% positive
Split	408 train / 102 test by contract, seed 42; re-run: 81 of the 408 for validation
Contract length	median 33,143 chars, max 338,211
Clause length	median 196 chars, p90 587, p99 1,330
Risk table	8 High / 22 Medium / 11 Low

Presence (4,182 test decisions)

Setup	micro-F1	Notes
First setup: TF-IDF / transformer / AND	0.775 / 0.692 / 0.779	AND accuracy 85.65%; AND vs TF-IDF a tie
Re-run, tuned for F1: TF-IDF / transformer / AND	0.785 / 0.797 / 0.809	AND accuracy 88.43%; lead +0.024 [+0.011, +0.037]
Re-run, recall-first: transformer (app) / averaging	0.757 / 0.783	High-risk found 90.3% / 86.9%
Control, 256 tokens tuned: transformer	0.761	loses to tuned TF-IDF by 0.024
Window fix alone (512 vs 256)	+0.024 untuned, +0.036 tuned	High-risk found +8.0 points
Macro-F1, 34 common categories	0.711 (re-run transformer) vs 0.673 (tuned TF-IDF)	

Truncation (first setup, 256 tokens): question 53 tokens; 49.3% of each window read (about 986 characters); about a third of each contract never read; 23.2% of positive training windows had the clause after the cut (4,875 of 21,028).

Risk levels (176 High-risk clauses): first setup AND misses 36.9%, transformer 15.9%, OR 14.2%; re-run app setting misses 9.7% (17), balanced AND 46.0% (81).

End to end: app with retrained span model 59.6% [57.0, 62.2] (first span model 22.1% [19.7, 24.6]; first setup 22.0%); right window 82.6% (was 69.1%); span OK given the right window 76.4% (first span model 28.0%; 76.2% / 76.8% in first / second half, was 40.7% / 5.4%). Quoted paragraph contains at least half the clause: 77.2% of detected, 71.7% of all real clauses.

Calibration: raw ECE 0.206, isotonic on validation 0.016, Platt 0.084; Brier 0.195 to 0.091.

Span: first model on centred windows token-F1 0.764 [0.743, 0.782], exact match 35.5%, overlap at F1 ≥ 0.5 82.8%; on plain windows 0.366. Retrained model on plain windows 0.779 (77.7% at F1 ≥ 0.5); 46.2 min training, 11,967 chunks from 6,000 clauses; chosen on validation (63.4% vs 20.9%).

Summarizer: ROUGE-L 0.775 vs templates, 0.116 vs clause-specific, 0.115 template alone, 0.299 untrained; 100% template copies, 70% right template.

Robustness (first setup): full data -0.0004; seed SD 0.0090 (transformer), 0.0015 (AND); split SD 0.0063; pooling best $\text{max@0.90} = 0.7529$; threshold 0.9 gives 0.753; retrieval ceiling 64.0%.

Training: first-setup presence ~50 min (256 tokens); re-run presence 191 min train + 198 min scoring (512 tokens, epoch 2 of 3); span ~25 min; summarizer 8-13 min CPU.

Longformer (M4): step 258 s / 14.71 GB at batch 1; ~136 days for our schedule; 454 s vs 10.6 s per median contract.

Clausify: median 64.4 s, worst 87.1 s, 1.93 GB; 30-window cap (38.4% of CUAD longer); default recall-first transformer; 45 tests (log in `tests/test_output.log`); demo contract 27 types (5 High, 13 Medium, 9 Low).